

Phase 1: Reconnaissance & Scope Guard

1. Executive Overview & Purpose

Phase 1 (Reconnaissance & Scope Guard) establishes the defensive boundary and asset discovery layer of the Automated Penetration Testing Orchestrator pipeline. Operating before any assessment traffic is transmitted, Phase 1 strictly enforces defined Rules of Engagement (RoE) by validating target IP addresses and CIDR ranges against authorized boundaries. Once verified, it conducts non-blocking port scanning and service fingerprinting, filtering out inactive ports to emit standardized, machine-readable JSON records (`scan_result.json`) that serve as clean input for Phase 2 correlation.

2. File Directory Breakdown & Component Purposes

The Phase 1 architecture is located within the `phase1_recon/` directory:

2.1 `phase1_recon/scope_guard.py`

- **Purpose:** Acts as the gatekeeper and RoE enforcement engine, preventing unauthorized scans or traffic dispatch to out-of-scope networks.
- **Core Responsibilities:**
 - Ingests target IP addresses and parses authorized CIDR ranges using `ipaddress.ip_network(..., strict=False)`.
 - Verifies whether the target IP falls strictly within authorized whitelists (e.g., `127.0.0.1/32`, `192.168.0.0/16`, `10.0.0.0/8`).
 - Blocks non-routable, multicast, and link-local addresses (such as cloud metadata endpoints like `169.254.169.254`) to prevent internal boundary violations.

2.2 `phase1_recon/models.py`

- **Purpose:** Enforces strict schemas and data contracts using Python Data Classes for all discovered discovery artifacts.
- **Core Responsibilities:**
 - Defines structured models: `ServiceInfo` (name, banner, version), `PortResult` (port number, protocol, state, service metadata), and `HostScanResult` (target IP, open port collection, UTC timestamp).
 - Automatically filters out closed or filtered ports (`state == "open"`), ensuring downstream phases receive concise data without unnecessary bloat.

Phase 1: Reconnaissance & Scope Guard

- Provides `.to_dict()` serialization for standardized JSON persistence.

2.3 `phase1_recon/scanner.py`

- **Purpose:** Serves as the asynchronous discovery and service banner extraction engine.
- **Core Responsibilities:**
 - Interfaces with the industry-standard python-nmap engine.
 - Encapsulates blocking Nmap subprocesses inside `asyncio.to_thread()` to prevent stalling the main asynchronous event loop.
 - Executes optimized scan parameters (`-sT -sV -Pn -n -T4`) to retrieve service versions while ensuring compatibility across unprivileged environments and Docker container loopback forwards.

2.4 `phase1_recon/main.py`

- **Purpose:** Serves as the primary Command-Line Interface (CLI) entry point for Phase 1 (`py -3 -m phase1_recon.main`).
- **Core Responsibilities:**
 - Parses command-line arguments (`--target`, `--ports`, `--allowed-cidrs`).
 - Triggers ScopeGuard validation before initiating network activities.
 - Invokes Scanner and writes structured results directly to `data/scanner_results/<target>.json`.

3. Critical Engineering Points in the Code

1. Non-Blocking Asynchronous Concurrency via `asyncio.to_thread`

Nmap CLI execution is inherently synchronous and process-heavy. Wrapping execution in `asyncio.to_thread()` allows the orchestrator to run port enumeration in worker threads while preserving event loop responsiveness for higher-level orchestration, logging, and future UI telemetry.

2. Strict RoE Scope Enforcement

Rather than performing simple string matching on hostnames or IP addresses, ScopeGuard utilizes Python's native `ipaddress` library to perform mathematical subnet containment checks, proactively blocking scanning if a target lies outside authorized CIDR boundaries.

3. Unprivileged TCP Connect Scanning (`-sT`)

Standard SYN stealth scans (`-sS`) require raw socket privileges (Administrator/root)

Phase 1: Reconnaissance & Scope Guard

and fail to traverse Windows userland Docker port proxying. Utilizing TCP Connect (-sT) guarantees reliable socket completion against mapped containers without elevated OS permissions.

4. Data Normalization at Ingestion

In older iterations, scanners exported every scanned port regardless of state. models.py enforces a strict gate where only verified open ports with corresponding service definitions are persisted, preventing inflated JSON files from choking Phase 2 correlation.

5. Dynamic Root Resolution

All Phase 1 modules dynamically resolve PROJECT_ROOT = Path(__file__).resolve().parent.parent and insert it into sys.path[0], ensuring cross-directory package execution (python -m phase1_recon.main) functions consistently without import resolution errors.

4. Problems Encountered and Technical Solutions

Problem Faced	Cause	Solution
Bloated JSON Handoff Files	Early scanner implementations recorded all scanned ports (open, closed, filtered), generating massive, noisy JSON outputs.	Updated models.py to filter strictly for state == "open", reducing JSON file sizes and eliminating irrelevant downstream processing.
Windows Socket WinError 10049	Scanning 0.0.0.0 caused the Windows network stack to fail because 0.0.0.0 is a non-routable listening meta-address, not a valid connection destination.	Restricted scan configurations and user input validation to routable loopback (127.0.0.1) or specific subnet addresses.
Unknown Service Banners on Custom Ports	Early custom socket fingerprinters (fingerprinter.py) relied on hardcoded ports (80, 443), resulting in missing service metadata for microservices on arbitrary ports (e.g., port 5000).	Deprecated custom socket fingerprinters in favor of python-nmap utilizing Nmap's comprehensive nmap-service-probes database (-sV).

Phase 1: Reconnaissance & Scope Guard

Event Loop Blocking During Long Scans	Direct execution of Nmap processes blocked the single-threaded Python asynchronous event loop.	Wrapped Nmap subprocess calls inside <code>asyncio.to_thread()</code> , delegating long-running scan execution to dedicated thread pools.
PortScannerError Binary Missing	Running the scanner failed because only the Python wrapper library was present, without the underlying system Nmap binary.	Installed the official Nmap executable on the host system, added its binary path to the system PATH, and configured system validation checks.

5. Rationale for Architecture, Tools & Technology Choices**1. Why python-nmap over Custom Socket Fingerprinting**

Maintaining a proprietary regex and socket handshake database across thousands of software versions is inefficient and error-prone. Adopting Nmap provides battle-tested service probe heuristics, protocol decoding, and standard version extraction out of the box.

2. Why Asynchronous Architecture (asyncio)

Network assessments involve significant I/O wait times. Using Python's asyncio ecosystem allows the orchestrator to scale horizontally, enabling non-blocking execution across concurrent targets and real-time event streaming to future reporting and dashboard modules.

3. Why Strict Subnet Validation (ipaddress)

Automated security tooling carries substantial liability if misconfigured. Embedding a mandatory pre-execution scope guard guarantees compliance with legal Rules of Engagement and protects unintended internal systems or cloud metadata endpoints from accidental probes.

4. Why Decoupled Data Contracts (JSON File Exchange via data/)

Decoupling Phase 1 from Phase 2 via structured JSON files eliminates tight coupling between modules. This modular boundary allows Phase 1 to be modified, replaced, or run as a standalone CLI tool without affecting downstream correlation or validation components.